[Trung Nguyen] Userspace Debugging Toolkit: GDB, strace & Valgrind

Debugging Tools on Linux: GDB, Valgrind, strace

1. GDB – GNU Debugger

1.1. What is GDB?

GDB (GNU Debugger) is a debugger: it lets you run a program under your control, stop it anywhere, and "look inside" to see what the program is doing.

In essence, GDB sits between you and the program. It uses an operating system mechanism (`ptrace` on Linux) to pause the process and read or modify its memory and registers. Thanks to the debug information the compiler embeds when you use the `-g` flag, GDB can map machine code back to source lines, variable names, and function names.

1.2. Common commands

Command	Shortcut	Purpose	Example
`run`	`r`	Run the program from the beginning	`r arg1 arg2`

Command	Shortcut	Purpose	Example
`start`		Run and stop at the beginning of `main`	`start`
`break`	`b`	Set a breakpoint	`b main`, `b file.c:25`
`continue`	`c`	Continue to the next breakpoint	`c`
`next`	`n`	Execute the next line, **without** entering functions	`n`, `n 5`
`step`	`s`	Execute the next line, **stepping into** functions	`s`
`finish`	`fin`	Run until the current function returns	`finish`
`until`	`u`	Run until past the current line (exit a loop)	`u`, `u 40`
`print`	`p`	Print the value of a variable/expression	`p x`, `p *ptr`, `p/x n`
`display`		Automatically print a variable at every stop	`display i`
`watch`		Stop when a variable changes	`watch count`
`backtrace`	`bt`	Show the call stack (which function called which)	`bt`, `bt full`
`frame`	`f`	Switch to another frame in the stack	`f 2`
`up` / `down`		Move up / down one frame	`up`
`info locals`	`i lo`	Print all local variables	`info locals`
`info args`		Print the arguments of the current function	`info args`
`info breakpoints`	`i b`	List breakpoints	`i b`
`delete`	`d`	Delete a breakpoint	`d 2`, `d` (delete all)
`list`	`l`	Show source code around the current position	`l`, `l func`
`set var`		Change a variable's value at runtime	`set var x = 10`
`layout src`		Open a split view showing source code	`layout src`
`thread apply all bt`		Show the backtrace of every thread	`thread apply all bt`
`quit`	`q`	Exit GDB	`q`

Command	Shortcut	Purpose	Example
Enter		Repeat the last command	(press Enter)

> [!NOTE] The most commonly confused pair is `next` and `step`. On a line `foo();`, `next` runs all of `foo` and stops at the following line, whereas `step` jumps to the first line inside `foo`. If you accidentally `step` into a function you don't care about, use `finish` to get back out.

1.3. Core dumps

1.3.1. What is a core dump and why does it matter?

A **core dump** is a "snapshot" of the entire state of a process at the moment it died: memory contents (stack, heap, global variables), register values, thread information, and so on. The operating system writes this file when a program is terminated by certain signals, such as:

Signal	Common cause
`SIGSEGV`	Invalid memory access (segfault)
`SIGABRT`	Call to `abort()`, failed `assert`
`SIGFPE`	Arithmetic error, e.g. division by zero

Its main role is **debugging after the failure has already happened** (*post-mortem debugging*). This is especially important when:

- The bug occurs on a production server.
- The bug is random and hard to reproduce.
- The program runs for a very long time before crashing.

Instead of waiting for the bug to recur under GDB, you just open the core file and immediately see where the program died, what the call stack looked like, and what values the variables held.

> [!NOTE] A core dump is a **static** state: you can inspect everything, but you cannot keep running (`next`, `continue`).

1.3.2. Generating a core dump on Linux

Step 1: Remove the core size limit in the current shell

```
ulimit -c unlimited
```

> This only applies to the current shell session.

Step 2: Find out where the core file is written

The location is controlled by:

```
cat /proc/sys/kernel/core_pattern
```

- If the result is a path/name such as `core` or `core.%e.%p`: the file will be placed in the program's working directory.
- If it starts with `|/usr/lib/systemd/systemd-coredump` (common on Ubuntu, Fedora...): cores are managed by systemd (view them with `coredumpctl`).

Change the pattern:

```
sudo sysctl -w kernel.core_pattern=/tmp/core.%e.%p
```

Specifier	Meaning
`%e`	Program name
`%p`	Process PID

Step 3: Run and catch the crash

```
gcc -g -O0 prog.c -o prog     # Compile with -g, optimizations off
./prog                        # Run and trigger the bug -> core dump is produced
gdb ./prog core               # Open GDB with the generated core dump
```

Inside GDB:

```
(gdb) bt             # Show the call stack at the time of the crash
(gdb) frame 1        # Switch to frame 1
(gdb) info locals    # Show local variables
```

1.4. Hands-on examples

1.4.1. Watching a variable change

Source code (`variable_change_bug.c`):

```
#include <stdio.h>

int counter = 10;

void foo(void)
{
    counter = 20;
}

void bar(void)
{
    counter = 50;
}

int main(void)
{
    foo();
    bar();

    printf("counter = %d\n", counter);

    return 0;
}
```

Steps:

1. Set a breakpoint on `main` and run the program:

   ```
   (gdb) break main
   (gdb) run
   ```

2. Set a watchpoint to track changes to `counter`:

   ```
   (gdb) watch counter
   ```

3. Let the program continue:

   ```
   (gdb) continue
   ```

Output:

```
(gdb) continue
Continuing.

Hardware watchpoint 2: counter

Old value = 10
New value = 20
foo () at variable_change_bug.c:8
8       }
(gdb) continue
Continuing.

Hardware watchpoint 2: counter

Old value = 20
New value = 50
bar () at variable_change_bug.c:13
13      }
```

> [!TIP] A watchpoint tells you exactly **when**, **where**, and the **old/new value** every time a variable changes.

1.4.2. The program crashes but you don't know on which line

Source code (`check_function_bug.c`):

```
#include <stdio.h>

void level3(void)
{
    int *p = NULL;

    *p = 100;
}

void level2(void)
```

```
{
    level3();
}

void level1(void)
{
    level2();
}

int main(void)
{
    level1();

    return 0;
}
```

Steps:

1. Compile and run the program to produce a core dump, then open it with GDB:

   ```
   gcc -g -O0 check_function_bug.c -o check_function_bug
   ./check_function_bug
   gdb ./check_function_bug core.function_bug.18806
   ```

   > The core file name depends on `core_pattern` and the process PID.

2. View the call stack at the time of the crash:

   ```
   (gdb) bt
   ```

   Output:

   ```
   #0  0x00006453f6ebc13d in level3 () at check_function_bug.c:7
   #1  0x00006453f6ebc153 in level2 () at check_function_bug.c:12
   #2  0x00006453f6ebc163 in level1 () at check_function_bug.c:17
   #3  0x00006453f6ebc173 in main () at check_function_bug.c:22
   ```

3. Switch to the crashing frame and inspect local variables:

   ```
   (gdb) frame 0
   #0  0x00006453f6ebc13d in level3 () at check_function_bug.c:7
   7       *p = 100;
   ```

```
(gdb) info locals
p = 0x0
```

Analysis: The program crashes at **line 7** in `level3()` because pointer `p` is `0x0` (`NULL`) but is still dereferenced.

Fix: Make `p` point to valid memory before writing:

```
void level3(void)
{
    int x;
    int *p = &x;

    *p = 100;
}
```

1.4.3. Analyzing a deadlock

Source code (`deadlock_thread.c`):

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    pthread_mutex_lock(&mutex1);
    sleep(1);
    pthread_mutex_lock(&mutex2);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&mutex2);
```

```
    sleep(1);
    pthread_mutex_lock(&mutex1);

    return NULL;
}

int main(void)
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Steps:

1. Compile and run the program in GDB. When it hangs, press `Ctrl + C` to pause it:

   ```
   gcc -g -O0 -pthread deadlock_thread.c -o deadlock_thread
   gdb ./deadlock_thread
   ```

   ```
   (gdb) run
   ^C
   ```

   > If the program is already running, you can attach GDB with `gdb -p <PID>`.

2. Show the backtrace of **all** threads:

   ```
   (gdb) thread apply all bt
   ```

Output:

```
Thread 3 (Thread 0x7ffff73fe6c0 (LWP 27492) "deadlock_thread"):
#0  futex_wait (private=0, expected=2, futex_word=0x555555558040 <mutex1>) at
../sysdeps/nptl/futex-internal.h:146
```

```
#1  __GI___lll_lock_wait (futex=futex@entry=0x555555558040 <mutex1>, private=0) at
./nptl/lowlevellock.c:49
#2  0x00007ffff7ca01e1 in lll_mutex_lock_optimized (mutex=0x555555558040 <mutex1>) at
./nptl/pthread_mutex_lock.c:48
#3  ___pthread_mutex_lock (mutex=0x555555558040 <mutex1>) at ./nptl/pthread_mutex_lock.c:93
#4  0x0000555555555240 in thread2 (arg=0x0) at deadlock_thread.c:21
#5  0x00007ffff7c9cb84 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#6  0x00007ffff7d29ecc in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78

Thread 2 (Thread 0x7ffff7bff6c0 (LWP 27491) "deadlock_thread"):
#0  futex_wait (private=0, expected=2, futex_word=0x555555558080 <mutex2>) at
../sysdeps/nptl/futex-internal.h:146
#1  __GI___lll_lock_wait (futex=futex@entry=0x555555558080 <mutex2>, private=0) at
./nptl/lowlevellock.c:49
#2  0x00007ffff7ca01e1 in lll_mutex_lock_optimized (mutex=0x555555558080 <mutex2>) at
./nptl/pthread_mutex_lock.c:48
#3  ___pthread_mutex_lock (mutex=0x555555558080 <mutex2>) at ./nptl/pthread_mutex_lock.c:93
#4  0x0000555555555201 in thread1 (arg=0x0) at deadlock_thread.c:12
#5  0x00007ffff7c9cb84 in start_thread (arg=<optimized out>) at ./nptl/pthread_create.c:447
#6  0x00007ffff7d29ecc in clone3 () at ../sysdeps/unix/sysv/linux/x86_64/clone3.S:78

Thread 1 (Thread 0x7ffff7faa740 (LWP 27465) "deadlock_thread"):
#0  0x00007ffff7c98e51 in __futex_abstimed_wait_common64 (private=128, cancel=true,
abstime=0x0, op=265, expected=27491, futex_word=0x7ffff7bff990) at ./nptl/futex-internal.c:57
#1  __futex_abstimed_wait_common (cancel=true, private=128, abstime=0x0, clockid=0,
expected=27491, futex_word=0x7ffff7bff990) at ./nptl/futex-interna
--Type <RET> for more, q to quit, c to continue without paging--Quit
```

How to read the backtrace (using Thread 3 as an example):

Frame	Meaning
`#0` – `#3`	Inside glibc: `pthread_mutex_lock` → `lll_lock_wait` → `futex_wait`, i.e. the thread is **waiting to acquire a mutex**.
`#4`	The first frame belonging to your `.c` file. **This is the line to focus on**: it shows the code responsible for the problem.
`#5` – `#6`	Thread startup (`start_thread`, `clone3`). Always present, unrelated to the bug.

Thread state summary:

Thread	LWP	Your code (frame `#4`)	What it's doing (`#0` – `#3`)	Waiting for
3	27492	`thread2()` line 21	`pthread_mutex_lock` → `futex_wait`	`mutex1`
2	27491	`thread1()` line 12	`pthread_mutex_lock` → `futex_wait`	`mutex2`
1	27465	`main()` (truncated, will be `pthread_join`)	`futex_wait` with `expected=27491`	Thread 27491 to exit

Analysis:

- `thread1` holds `mutex1` and waits for `mutex2`.
- `thread2` holds `mutex2` and waits for `mutex1`.
- `main` waits for `thread1` (LWP 27491) to finish via `pthread_join`.

→ The two threads wait on each other in a cycle (*circular wait*), and neither can make progress ⇒ **deadlock**.

Fix: Make every thread acquire the mutexes in **the same order**, e.g. always `mutex1` before `mutex2`, and `unlock` when done:

```
void *thread1(void *arg)
{
    pthread_mutex_lock(&mutex1);
    sleep(1);
    pthread_mutex_lock(&mutex2);

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&mutex1);
    sleep(1);
    pthread_mutex_lock(&mutex2);

    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
```

```
    return NULL;
}
```

> [!TIP] Always `unlock` a mutex when you're done with it. In the original version, the threads `return` without unlocking, which is another latent bug.

---

2. Valgrind

2.1. What is Valgrind?

Valgrind is a **dynamic analysis** toolkit for Linux (dynamic analysis means checking a program while it is running).

It works by running your program on a **virtual CPU** emulated by Valgrind itself. Every machine instruction passes through Valgrind before it executes, so Valgrind can observe every memory read and write, and every allocation and deallocation. This technique is called **dynamic binary instrumentation**: inserting checking code into the executable at runtime.

Consequences of this design:

- **No need to recompile with special flags.** You should only add `-g` so errors are reported with file names and line numbers.
- **It runs slowly**, roughly 10 to 50 times slower than normal, and uses extra RAM.

Main tools in the Valgrind suite:

Tool	Purpose
`memcheck` (default)	Detects memory errors: out-of-bounds reads/writes, use-after-free, uninitialized variables, memory leaks
`helgrind`	Detects thread errors: data races, lock order violations (potential deadlocks)

2.2. Installation and usage

Install:

```
sudo apt install valgrind
valgrind --version
```

Compile the program, then run it under Valgrind:

```
gcc -g mem_bug.c -o mem_bug
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./mem_bug
```

Option	Purpose
`--leak-check=full`	Show which line `malloc`'d the leaked memory
`--show-leak-kinds=all`	Show every kind of leak
`--track-origins=yes`	For uninitialized values, show where they originated
`--log-file=vg.log`	Write the output to a file (useful when output is long)
`--tool=helgrind`	Switch to the thread error detector

2.3. Example 1: memory errors with Memcheck

Source code (`mem_bug.c`):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    int *arr = malloc(5 * sizeof(int));
    for (int i = 0; i <= 5; i++)
        arr[i] = i;

    int x;
    if (x > 10)
        printf("x lon\n");

    char *name = malloc(16);
    strcpy(name, "trung");
```

```
    free(name);
    printf("%s\n", name);

    int *leak = malloc(100);
    leak[0] = 1;

    free(arr);
    return 0;
}
```

This program contains 4 bugs:

Line	Bug
8 – 9	The loop uses `<=`, so it writes to `arr[5]`, past the end of the 5-element array
11 – 12	Uninitialized variable `x` is used in an `if` condition
17 – 18	`name` is read after being `free`d (*use-after-free*)
20	`leak` is `malloc`'d but never `free`d (memory leak)

Output when run normally:

```
fSA
```

The program doesn't crash, but it prints garbage because it reads freed memory. The other bugs are completely "silent". This is the kind of bug GDB has a hard time catching.

Log when run under Valgrind:

```
trung@trungnm:valgrind$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes
./mem_bug
==32119== Memcheck, a memory error detector
==32119== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==32119== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==32119== Command: ./mem_bug
==32119==
==32119== Invalid write of size 4
==32119==    at 0x1091C3: main (mem_bug.c:9)
==32119==  Address 0x4a7e054 is 0 bytes after a block of size 20 alloc'd
==32119==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x10919E: main (mem_bug.c:7)
==32119==
```

```
==32119== Conditional jump or move depends on uninitialised value(s)
==32119==    at 0x1091D3: main (mem_bug.c:12)
==32119==  Uninitialised value was created by a stack allocation
==32119==    at 0x109189: main (mem_bug.c:6)
==32119==
==32119== Invalid read of size 1
==32119==    at 0x484F226: strlen (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x48EFCDC: puts (ioputs.c:35)
==32119==    by 0x109219: main (mem_bug.c:18)
==32119==  Address 0x4a7e0a0 is 0 bytes inside a block of size 16 free'd
==32119==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x10920D: main (mem_bug.c:17)
==32119==  Block was alloc'd at
==32119==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x1091ED: main (mem_bug.c:15)
==32119==
==32119== Invalid read of size 1
==32119==    at 0x484F234: strlen (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x48EFCDC: puts (ioputs.c:35)
==32119==    by 0x109219: main (mem_bug.c:18)
==32119==  Address 0x4a7e0a1 is 1 bytes inside a block of size 16 free'd
==32119==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x10920D: main (mem_bug.c:17)
==32119==  Block was alloc'd at
==32119==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x1091ED: main (mem_bug.c:15)
==32119==
==32119== Invalid read of size 1
==32119==    at 0x48FD74B: _IO_default_xsputn (genops.c:399)
==32119==    by 0x48FD74B: _IO_default_xsputn (genops.c:370)
==32119==    by 0x48FBBC9: _IO_new_file_xsputn (fileops.c:1265)
==32119==    by 0x48FBBC9: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1197)
==32119==    by 0x48EFD3C: puts (ioputs.c:40)
==32119==    by 0x109219: main (mem_bug.c:18)
==32119==  Address 0x4a7e0a0 is 0 bytes inside a block of size 16 free'd
==32119==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x10920D: main (mem_bug.c:17)
==32119==  Block was alloc'd at
==32119==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x1091ED: main (mem_bug.c:15)
```

```
==32119==
trung
==32119==
==32119== HEAP SUMMARY:
==32119==     in use at exit: 100 bytes in 1 blocks
==32119==   total heap usage: 4 allocs, 3 frees, 1,160 bytes allocated
==32119==
==32119== 100 bytes in 1 blocks are definitely lost in loss record 1 of 1
==32119==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==32119==    by 0x109223: main (mem_bug.c:20)
==32119==
==32119== LEAK SUMMARY:
==32119==    definitely lost: 100 bytes in 1 blocks
==32119==    indirectly lost: 0 bytes in 0 blocks
==32119==      possibly lost: 0 bytes in 0 blocks
==32119==    still reachable: 0 bytes in 0 blocks
==32119==         suppressed: 0 bytes in 0 blocks
==32119==
==32119== For lists of detected and suppressed errors, rerun with: -s
==32119== ERROR SUMMARY: 14 errors from 6 contexts (suppressed: 0 from 0)
```

> [!NOTE] Under Valgrind the program prints `trung` instead of garbage, because Valgrind holds on to freed memory for a while in order to detect use-after-free. The bug is still there; it just shows up differently.

Reading the log

Error 1: `Invalid write of size 4` – out-of-bounds write

```
Invalid write of size 4
   at main (mem_bug.c:9)
 Address 0x4a7e054 is 0 bytes after a block of size 20 alloc'd
   by main (mem_bug.c:7)
```

- `write of size 4`: writing 4 bytes, i.e. one `int`.
- `0 bytes after a block of size 20`: writing right after a 20-byte block, i.e. `arr[5]`.
- `alloc'd at mem_bug.c:7`: that block is `arr`.

→ **Conclusion:** the loop on line 8 uses `<=` instead of `<`.

Error 2: `Conditional jump or move depends on uninitialised value(s)` – use of an uninitialized variable

```
Conditional jump or move depends on uninitialised value(s)
   at main (mem_bug.c:12)
 Uninitialised value was created by a stack allocation
   at main (mem_bug.c:6)
```

- Line 12 has an `if` statement that decides based on a garbage value.
- `stack allocation` means it's a local variable of `main`, namely `x`.

> The `Uninitialised value was created by...` part only appears with `--track-origins=yes`.

Error 3: `Invalid read` – use after free

```
Invalid read of size 1
   at strlen
   by puts (ioputs.c:35)
   by main (mem_bug.c:18)
 Address 0x4a7e0a0 is 0 bytes inside a block of size 16 free'd
   by main (mem_bug.c:17)
 Block was alloc'd at
   by main (mem_bug.c:15)
```

Valgrind reports the full "life story" of the block: `malloc` on line 15, `free` on line 17, then still read on line 18. The error repeats several times because `printf`/`puts` reads the string byte by byte.

Error 4: memory leak

```
HEAP SUMMARY:
    in use at exit: 100 bytes in 1 blocks
  total heap usage: 4 allocs, 3 frees, 1,160 bytes allocated


100 bytes in 1 blocks are definitely lost in loss record 1 of 1
   at malloc
   by main (mem_bug.c:20)
```

- At exit, 100 bytes were still not freed.
- `malloc` was called 4 times but `free` only 3 times.
- It points directly to the `malloc` on line 20 that was never freed.

Leak kinds in `LEAK SUMMARY`:

Kind	Meaning	Severity
`definitely lost`	No pointer to the block remains; definitely leaked	Must fix
`indirectly lost`	Lost because of a "definitely lost" block (e.g. nodes of a linked list whose head was lost)	Fixing the parent block fixes these
`possibly lost`	Only a pointer into the middle of the block remains	Should investigate
`still reachable`	A pointer still exists; it just wasn't freed before exit	Acceptable

`ERROR SUMMARY: 14 errors from 6 contexts`

- **14 errors**: total number of error occurrences detected.
- **6 contexts**: number of distinct places in the code with errors.

> [!TIP] The goal is `ERROR SUMMARY: 0 errors` and `definitely lost: 0 bytes`.

Fixed code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void)
{
    int *arr = malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++)         /* Fix: < instead of <= */
        arr[i] = i;

    int x = 0;                          /* Fix: initialize the variable */
    if (x > 10)
        printf("x lon\n");

    char *name = malloc(16);
    strcpy(name, "trung");
    printf("%s\n", name);               /* Fix: use before free */
    free(name);
```

```
    int *leak = malloc(100);
    leak[0] = 1;
    free(leak);                          /* Fix: release the memory */

    free(arr);
    return 0;
}
```

2.4. Example 2: thread errors with Helgrind

Source code (`thread_bug.c`):

```
#include <pthread.h>

pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;

void *thread1(void *arg)
{
    pthread_mutex_lock(&mutex1);
    pthread_mutex_lock(&mutex2);
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    counter++;
    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&mutex2);
    pthread_mutex_lock(&mutex1);
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
    counter++;
```

```
    return NULL;
}

int main(void)
{
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

This code has two bugs:

- **Potential deadlock:** the two threads lock `mutex1` and `mutex2` in opposite orders. Most runs finish normally, but if the two threads interleave at just the wrong moment, the program hangs.
- **Data race:** both threads increment `counter` **without holding any lock**.

Run with Helgrind:

```
gcc -g -pthread thread_bug.c -o thread_bug
valgrind --tool=helgrind ./thread_bug
```

Resulting log:

```
trung@trungnm:valgrind$ valgrind --tool=helgrind ./thread_bug
==33577== Helgrind, a thread error detector
==33577== Copyright (C) 2007-2017, and GNU GPL'd, by OpenWorks LLP et al.
==33577== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==33577== Command: ./thread_bug
==33577==
==33577== ---Thread-Announcement------------------------------------------
==33577==
==33577== Thread #3 was created
==33577==    at 0x499ACB3: clone (clone.S:76)
==33577==    by 0x499AE32: __clone_internal_fallback (clone-internal.c:64)
==33577==    by 0x499AE32: __clone_internal (clone-internal.c:109)
==33577==    by 0x490D62F: create_thread (pthread_create.c:297)
==33577==    by 0x490E284: pthread_create@@GLIBC_2.34 (pthread_create.c:836)
```

```
==33577==    by 0x4854975: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x1092E1: main (thread_bug.c:31)
==33577==
==33577== ----------------------------------------------------------------
==33577==
==33577== Thread #3: lock order "0x10C040 before 0x10C080" violated
==33577==
==33577== Observed (incorrect) order is: acquisition of lock at 0x10C080
==33577==    at 0x48512DC: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x109249: thread2 (thread_bug.c:19)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==
==33577==  followed by a later acquisition of lock at 0x10C040
==33577==    at 0x48512DC: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x109258: thread2 (thread_bug.c:20)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==
==33577== Required order was established by acquisition of lock at 0x10C040
==33577==    at 0x48512DC: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x1091E7: thread1 (thread_bug.c:9)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==
==33577==  followed by a later acquisition of lock at 0x10C080
==33577==    at 0x48512DC: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x1091F6: thread1 (thread_bug.c:10)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==
==33577==  Lock at 0x10C040 was first observed
==33577==    at 0x48512DC: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x1091E7: thread1 (thread_bug.c:9)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
```

```
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==  Address 0x10c040 is 0 bytes inside data symbol "mutex1"
==33577==
==33577==  Lock at 0x10C080 was first observed
==33577==    at 0x48512DC: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x1091F6: thread1 (thread_bug.c:10)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==  Address 0x10c080 is 0 bytes inside data symbol "mutex2"
==33577==
==33577==
==33577== ---Thread-Announcement----------------------------------------
==33577==
==33577== Thread #2 was created
==33577==    at 0x499ACB3: clone (clone.S:76)
==33577==    by 0x499AE32: __clone_internal_fallback (clone-internal.c:64)
==33577==    by 0x499AE32: __clone_internal (clone-internal.c:109)
==33577==    by 0x490D62F: create_thread (pthread_create.c:297)
==33577==    by 0x490E284: pthread_create@@GLIBC_2.34 (pthread_create.c:836)
==33577==    by 0x4854975: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x1092C4: main (thread_bug.c:30)
==33577==
==33577== ----------------------------------------------------------------
==33577==
==33577== Possible data race during read of size 4 at 0x10C0A8 by thread #3
==33577== Locks held: none
==33577==    at 0x109277: thread2 (thread_bug.c:23)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==
==33577== This conflicts with a previous write of size 4 by thread #2
==33577== Locks held: none
==33577==    at 0x10921E: thread1 (thread_bug.c:13)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==  Address 0x10c0a8 is 0 bytes inside data symbol "counter"
==33577==
```

```
==33577== ------------------------------------------------------------
==33577==
==33577== Possible data race during write of size 4 at 0x10C0A8 by thread #3
==33577== Locks held: none
==33577==    at 0x109280: thread2 (thread_bug.c:23)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==
==33577== This conflicts with a previous write of size 4 by thread #2
==33577== Locks held: none
==33577==    at 0x10921E: thread1 (thread_bug.c:13)
==33577==    by 0x4854B7A: ??? (in /usr/libexec/valgrind/vgpreload_helgrind-amd64-linux.so)
==33577==    by 0x490DB83: start_thread (pthread_create.c:447)
==33577==    by 0x499ACC3: clone (clone.S:100)
==33577==  Address 0x10c0a8 is 0 bytes inside data symbol "counter"
==33577==
==33577==
==33577== Use --history-level=approx or =none to gain increased speed, at
==33577== the cost of reduced accuracy of conflicting-access information
==33577== For lists of detected and suppressed errors, rerun with: -s
==33577== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 12 from 12)
```

Reading the log

Step 1: Map addresses to mutexes

Helgrind reports errors by address, so first look up which variable each address belongs to. At the end of the report you'll find:

```
Address 0x10c040 is 0 bytes inside data symbol "mutex1"
Address 0x10c080 is 0 bytes inside data symbol "mutex2"
```

Address	Variable
`0x10C040`	`mutex1`
`0x10C080`	`mutex2`
`0x10C0A8`	`counter`

Step 2: `lock order violated` – potential deadlock

The line `lock order "0x10C040 before 0x10C080" violated` means: the established order is `mutex1` → `mutex2`, but some thread broke that order.

The *required order* is the order Helgrind saw first (established by `thread1`):

```
Required order was established by acquisition of lock at 0x10C040
   by thread1 (thread_bug.c:9)      ← thread1 locks mutex1 first
 followed by a later acquisition of lock at 0x10C080
   by thread1 (thread_bug.c:10)     ← then locks mutex2
```

The *observed (incorrect) order* is the violating order (by `thread2`):

```
Observed (incorrect) order is: acquisition of lock at 0x10C080
   by thread2 (thread_bug.c:19)     ← thread2 locks mutex2 first
 followed by a later acquisition of lock at 0x10C040
   by thread2 (thread_bug.c:20)     ← then locks mutex1
```

→ If the two threads interleave at the wrong moment (thread1 holds `mutex1`, thread2 holds `mutex2`), both will wait on each other forever ⇒ **deadlock**. Helgrind's strength is that it detects this **even when that particular run does not hang**.

Step 3: `Possible data race`

```
Possible data race during write of size 4 at 0x10C0A8 by thread #3
Locks held: none
   at thread2 (thread_bug.c:23)
This conflicts with a previous write of size 4 by thread #2
Locks held: none
   at thread1 (thread_bug.c:13)
 Address 0x10c0a8 is 0 bytes inside data symbol "counter"
```

- Both threads read/write `counter` (line 13 and line 23).
- `Locks held: none`: neither access holds a lock.
- `counter++` consists of three steps (read → add → write), so the two threads can overwrite each other's result.

Fixed code

```
void *thread1(void *arg)
{
    pthread_mutex_lock(&mutex1);
    pthread_mutex_lock(&mutex2);
```

```
    counter++;                          /* Fix: increment counter while holding the lock */
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}

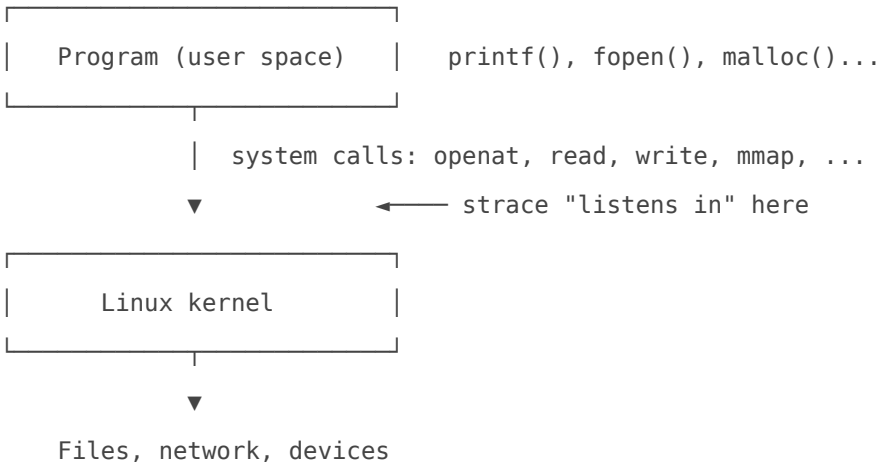
void *thread2(void *arg)
{
    pthread_mutex_lock(&mutex1);        /* Fix: same order mutex1 -> mutex2 */
    pthread_mutex_lock(&mutex2);
    counter++;
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    return NULL;
}
```

3. strace

3.1. What is strace?

strace is a Linux command-line tool that traces the **system calls** and **signals** a process makes and receives.

Programs don't talk to hardware directly; they "talk" to the operating system through system calls: opening files, reading/writing, allocating memory, creating processes, making network connections, and so on.



strace is very useful when **you don't have the source code**, or when you need to know how a program interacts with the system: a missing config file, permission denied, hanging while waiting on the network...

3.2. Installation

```
sudo apt update && sudo apt install strace -y
```

3.3. Basic usage

Trace a program from the moment it starts:

```
strace ls
```

Attach to a running process by PID:

```
strace -p <PROCESS_PID>
```

> You may need `sudo` if the process is not a child of the current shell (due to the system's `ptrace` restrictions).

Write the output to a file:

```
strace -f -o /tmp/trace_output.log ./my_program
```

3.4. Common options

Option	Purpose
`-p <PID>`	Attach to a running process
`-o <file>`	Write output to a file
`-f`	Also trace child processes and threads
`-e trace=openat,read,write`	Trace only the listed system calls
`-e trace=file` / `network` / `process`	Trace only a group of system calls
`-c`	Summarize call counts and time for each system call
`-t` / `-tt`	Print the timestamp of each system call

Option	Purpose
`-T`	Print the time spent in each system call
`-s <n>`	Maximum string length to print (default 32)

3.5. Reading the output

Each line has the form:

```
syscall_name(arguments, ...) = return_value
```

Example:

```
execve("/usr/bin/ls", ["ls"], 0x7ffd... /* 50 vars */) = 0
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
openat(AT_FDCWD, "config.ini", O_RDONLY) = -1 ENOENT (No such file or directory)
write(1, "file1  file2\n", 13)          = 13
exit_group(0)                           = ?
```

- `openat(...) = 3`: the file was opened successfully, returning file descriptor 3.
- `= -1 ENOENT (...)`: the call failed, with an error code. This is often the most important clue (e.g. the program is looking for its config file in the wrong place).
- `write(1, ...)`: writing to file descriptor 1, i.e. `stdout`.

---

4. Summary: which tool should I use?

Tool	Use when	Strengths
GDB	The program crashes or hangs and you need to inspect its state on the spot	Stop, step line by line, inspect variables, call stack, core dumps
Valgrind	The program seems to work but you suspect hidden memory or thread bugs	Finds "silent" bugs GDB can't see (leaks, use-after-free, data races, potential deadlocks)
strace	You need to know how the program interacts with the OS, or you don't have the source	Shows every file, network, and process operation, plus errors returned by the kernel

#LAB and Example lab

Revision #4
Created 2026-09-22 09:01:16 UTC by trungnguyen
Updated 2026-09-23 02:04:59 UTC by trungnguyen